

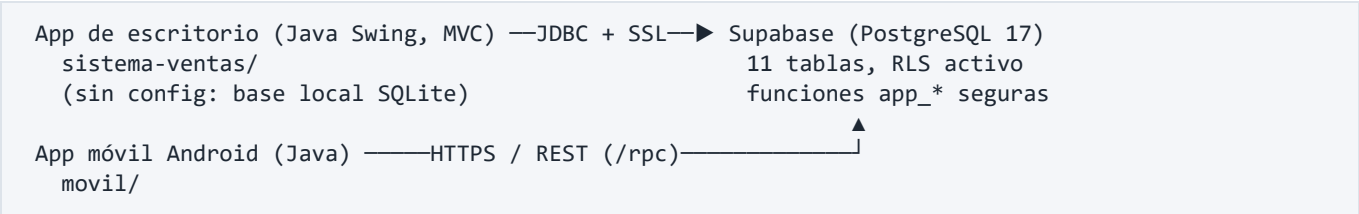
# Manual Técnico — Sistema de Ventas

Universidad Mariano Gálvez · Programación II

Autor: Josue Zetino · Repositorio: <https://github.com/jzetinob/sistema-ventas>

## 1. Visión general

El sistema tiene tres piezas que comparten los mismos datos:



| Pieza         | Tecnología                                                                                                                                                               | Carpeta                                            |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Escritorio    | Java 25, Swing (MDI con <code>JDesktopPane</code> ), Maven, NetBeans                                                                                                     | <code>sistema-ventas/</code>                       |
| Base de datos | PostgreSQL 17 en Supabase (nube) o SQLite (local, sin configuración)                                                                                                     | <code>sistema-ventas/src/main/resources/bd/</code> |
| Móvil         | Android (Java 17; minSdk 24, compileSdk 37, targetSdk 34). Solo usa lo que trae Android, más la librería oficial <code>SwipeRefreshLayout</code> . Se compila con Gradle | <code>movil/</code>                                |
| Documentación | Markdown                                                                                                                                                                 | <code>docs/</code> , <code>tareas/</code>          |

## 2. Requisitos de desarrollo

| Herramienta                    | Versión                                              |
|--------------------------------|------------------------------------------------------|
| JDK                            | 25 ( <code>maven.compiler.release 25</code> )        |
| Apache NetBeans                | 22 o superior (probado con 25)                       |
| Maven                          | el que incluye NetBeans                              |
| Android Studio                 | 2026.1 o superior (SDK de Android, API 37)           |
| Gradle / Android Gradle Plugin | 9.8 / 9.4.1 (incluidos: <code>movil/gradlew</code> ) |
| Cuenta de Supabase             | plan Free                                            |

Dependencias Maven ( `sistema-ventas/pom.xml` ):

| Dependencia                      | Uso                          |
|----------------------------------|------------------------------|
| org.xerial:sqlite-jdbc:3.47.1.0  | Base local SQLite            |
| org.postgresql:postgresql:42.7.4 | Base en la nube              |
| org.mindrot:jbcrypt:0.4          | Hash de contraseñas (bcrypt) |

### 3. Arquitectura de la app de escritorio

Patrón **MVC** en el paquete `com.josue.ventas` :

| Paquete     | Contenido                                                                                                                                                                                                                                                                                                                                                                                |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| modelo      | Entidades: <code>Persona</code> (abstracta) → <code>Cliente</code> , <code>Empleado</code> , <code>Proveedor</code> ; <code>Producto</code> , <code>Categoria</code> , <code>Factura</code> + <code>FacturaDetalle</code> , <code>Compra</code> + <code>CompraDetalle</code> , <code>Usuario</code>                                                                                      |
| dao         | Interfaces DAO y su implementación JDBC ( <code>...DAOSQLite</code> , que funcionan también con PostgreSQL), <code>ConexionBD</code> , <code>CopiadorBD</code> , <code>MigradorDatos</code> (CSV antiguos)                                                                                                                                                                               |
| controlador | Un controller por entidad, <code>Sesion</code> (usuario conectado), <code>Contraseñas</code> (hash), <code>RespaldoController</code>                                                                                                                                                                                                                                                     |
| vista       | Formularios ( <code>JInternalFrame</code> ), <code>FrmCatalogo</code> (base abstracta), <code>DlgLogin</code> , <code>FrmResumen</code> + <code>GraficoBarrasPanel</code> / <code>GraficoDonaPanel</code> (gráficos con <code>Graphics2D</code> ), <code>BarraBusqueda</code> , <code>FiltroTabla</code> , <code>ReporteHtml</code> , <code>Reportes</code> , <code>CampoBusqueda</code> |

Flujo de una operación: **vista** → **controlador** → **DAO** → **base de datos**. Las vistas nunca escriben SQL.

#### 3.1 Patrones de diseño aplicados

| Patrón                         | Dónde                                                                                                                                                                                                                                                        |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MVC</b>                     | Separación <code>vista</code> / <code>controlador</code> / <code>modelo</code> + <code>dao</code>                                                                                                                                                            |
| <b>Singleton</b>               | <code>ConexionBD</code> , cada <code>...DAOSQLite</code> , <code>Sesion</code> , <code>ApiSupabase</code> (móvil)                                                                                                                                            |
| <b>DAO</b>                     | Interfaces <code>ClienteDAO</code> , <code>ProductoDAO</code> ... con implementación intercambiable (CSV → SQLite/PostgreSQL) sin tocar las vistas                                                                                                           |
| <b>Template Method</b>         | <code>FrmCatalogo</code> arma la pantalla y define los pasos abstractos <code>guardar()</code> , <code>actualizar()</code> , <code>eliminar()</code> ... que implementan <code>FrmCategorias</code> , <code>FrmProveedores</code> , <code>FrmUsuarios</code> |
| <b>Herencia / polimorfismo</b> | <code>Persona</code> abstracta; <code>Empleado</code> y <code>Proveedor</code> sobrescriben <code>mostrarInformacion()</code> ; <code>ActividadBase</code> en Android                                                                                        |
| <b>Composición</b>             | <code>Factura</code> ◆— <code>FacturaDetalle</code> , <code>Compra</code> ◆— <code>CompraDetalle</code>                                                                                                                                                      |

#### 3.2 Selección del motor de base de datos

`ConexionBD` lee `sistema-ventas/config/bd.properties` al arrancar:

- **No existe** → SQLite en `datos/sistema_ventas.db` (crea el archivo y las tablas).

- `bd.motor=postgresql` → se conecta a Supabase y ejecuta `bd/esquema-postgresql.sql`, que se puede correr siempre porque es idempotente.

```
bd.motor=postgresql
bd.url=jdbc:postgresql://aws-0-us-east-1.pooler.supabase.com:5432/postgres
bd.usuario=postgres.<referencia-del-proyecto>
bd.clave=<contraseña de la base de datos>
```

`config/bd.properties` contiene la contraseña: está en `.gitignore` y **nunca** se sube a GitHub. En el repositorio solo está `bd.properties.ejemplo`.

Los DAO usan SQL estándar que funciona en ambos motores. Lo único específico de cada motor está en `ConexionBD`: la creación de tablas, las migraciones y el respaldo.

**Primera conexión a la nube:** si la base de Supabase está vacía y existe `datos/sistema_ventas.db` con datos, `CopiadorBD` los sube automáticamente. Conserva los `id` y ajusta las secuencias.

**Migraciones SQLite:** las bases creadas con versiones anteriores se actualizan solas ( `agregarColumnaSiFalta` ): `productos.existencia`, `productos.categoria_id`, `factura_detalle.producto_id`. `MigradorDatos` importa una sola vez los CSV de las primeras versiones ( `clientes.csv`, `productos.csv`, `facturas.csv`, `detalles.csv` ) si la base está vacía. Los DAO CSV ( `...DAOCsv` ) quedaron en el código como referencia del entregable de listas y archivos.

### 3.3 Detalles de implementación

- **Ventana MDI:** `FrmPrincipal` contiene un `JDesktopPane`, y cada formulario es un `JInternalFrame`. Si una ventana ya está abierta ( `isDisplayable()` ), se trae al frente en lugar de crear otra. El menú *Ventana* acomoda en cascada o mosaico, y minimiza o restaura todas.
- **Correlativos (FAC-0001, COM-0001):** se calculan como el mayor número existente más uno, consultando la tabla. No hay un contador aparte, así que no se repiten al reiniciar y no se "quemán" números al cerrar un formulario sin guardar.
- **Impresión:** `TicketFactura` implementa `java.awt.print.Printable` y dibuja el ticket con `Graphics2D`. `FrmVistaPreviaFactura` usa el mismo método `pintar(...)` para la vista previa y abre el diálogo de impresión del sistema ( `PrinterJob.printDialog()` ). Se imprime desde la factura en curso y desde el detalle de una factura guardada, sin librerías externas.
- **Buscador con autocompletado ( `CampoBusqueda` ):**
  - Es un `JTextField` con un menú emergente y una `JList`. Filtra por "contiene", sin distinguir mayúsculas; se navega con ↑/↓, Enter elige y Escape cierra.
  - Vuelve a leer su lista ( `setFuente` ) cada vez que recibe el foco, así aparecen los registros creados en otra ventana o desde el celular.
  - Si nada coincide, muestra "(sin coincidencias)".
  - Avisa a sus oyentes ( `addActionListener` ) al elegir un elemento. Es el patrón Observer.
- **Resumen con gráficos ( `FrmResumen` ):** `ResumenController` toma una "foto" de las facturas y los productos (a través de sus controllers, así funciona con los dos motores) y calcula los indicadores, las ventas de cada uno de los últimos 7 días (con ceros), el top 5 del mes y la existencia baja.

`GraficoBarrasPanel` y `GraficoDonaPanel` sobrescriben `paintComponent()` y dibujan con `Graphics2D` (`fillRoundRect`, `Arc2D`). Es la misma idea que `onDraw()` con `Canvas` en Android.

- **Búsqueda en tablas ( `FiltroTabla`, `BarraBusqueda` ):** usan `TableRowSorter` y `RowFilter`. Cuando la tabla está filtrada, la fila seleccionada se traduce con `convertRowIndexToModel`.

## 4. Base de datos

### 4.1 Tablas

Hay 10 tablas del sistema, más `sesiones_app` para el móvil (solo en PostgreSQL). El diagrama completo está en `diagramas/diagrama-entidad-relacion.md`.

| Tabla                        | Descripción                              | Llaves foráneas                                                          |
|------------------------------|------------------------------------------|--------------------------------------------------------------------------|
| <code>categorias</code>      | Clasificación de productos               | —                                                                        |
| <code>productos</code>       | Catálogo, precio y existencia            | <code>categoria_id</code> → <code>categorias</code> (SET NULL)           |
| <code>clientes</code>        | NIT único                                | —                                                                        |
| <code>empleados</code>       | Código único                             | —                                                                        |
| <code>proveedores</code>     | NIT único                                | —                                                                        |
| <code>usuarios</code>        | Usuario, hash de contraseña, rol, activo | —                                                                        |
| <code>facturas</code>        | Encabezado de venta                      | —                                                                        |
| <code>factura_detalle</code> | Líneas de la factura                     | <code>factura_id</code> (CASCADE), <code>producto_id</code> (SET NULL)   |
| <code>compras</code>         | Encabezado de compra                     | <code>proveedor_id</code> (RESTRICT), <code>usuario_id</code> (SET NULL) |
| <code>compra_detalle</code>  | Líneas de la compra                      | <code>compra_id</code> (CASCADE), <code>producto_id</code> (RESTRICT)    |
| <code>sesiones_app</code>    | Tokens de la app móvil (8 h)             | <code>usuario_id</code> (CASCADE)                                        |

En SQLite, las llaves foráneas se activan en cada conexión con `PRAGMA foreign_keys = ON`.

### 4.2 Transacciones

Estas operaciones se ejecutan completas o no se ejecutan ( `setAutoCommit(false)` + `commit` / `rollback` ):

- **Guardar factura:** inserta el encabezado y los detalles, y descuenta la existencia con `UPDATE ... WHERE existencia >= ?`. Si un producto no alcanza, se revierte todo.
- **Eliminar factura:** devuelve la existencia y borra la factura (los detalles se borran en cascada).
- **Guardar compra:** inserta el encabezado y los detalles, y suma la existencia.
- **Anular compra:** resta la existencia (se rechaza si ya no alcanza) y borra la compra.

## 5. Seguridad

| Medida                        | Implementación                                                                                                                                                                                                                |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inyección SQL                 | Todo dato del usuario viaja en <code>PreparedStatement</code> . Los únicos SQL armados con texto usan nombres de tablas y columnas fijos del código                                                                           |
| Contraseñas                   | <code>bcrypt</code> , costo 10 ( <code>Contrasenas</code> ). Los hash antiguos PBKDF2-SHA256 (120 000 iteraciones) siguen siendo válidos y se convierten a <code>bcrypt</code> en el siguiente inicio de sesión               |
| Inicio de sesión              | El mismo mensaje para usuario inexistente y contraseña incorrecta; cierre tras 3 intentos; <code>char[]</code> limpiado después de usarse                                                                                     |
| Roles                         | Administrador / Vendedor; el menú se ajusta al rol ( <code>FrmPrincipal.aplicarSesion</code> )                                                                                                                                |
| Reglas de negocio             | No es posible eliminarse a uno mismo; siempre queda al menos un administrador activo                                                                                                                                          |
| Reportes                      | Todo dato se escapa ( <code>ReporteHtml.escapar</code> ) antes de escribirse en HTML                                                                                                                                          |
| Búsqueda                      | <code>Pattern.quote</code> : lo que escribe el usuario no se interpreta como expresión regular                                                                                                                                |
| Credenciales                  | La contraseña de la base solo está en <code>config/bd.properties</code> (ignorado por Git)                                                                                                                                    |
| <b>API pública (Supabase)</b> | <b>RLS activo en las 11 tablas y sin políticas</b> , y permisos de tablas revocados a <code>anon / authenticated</code> . La API no puede leer ni escribir tablas. Solo puede ejecutar las funciones <code>app_*</code>       |
| Funciones móviles             | <code>SECURITY DEFINER</code> con <code>search_path</code> fijo; todas validan el token ( <code>app_usuario_de</code> ); el login espera 1 s tras un fallo; los tokens vencen a las 8 h y <code>app_logout</code> los elimina |

### 5.1 API de la app móvil

Todas se llaman con `POST https://<proyecto>.supabase.co/rest/v1/rpc/<función>` , el encabezado `apikey:<clave publishable>` y un cuerpo JSON.

| Función               | Parámetros                                                                    | Devuelve                                                                                    |
|-----------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| app_login             | p_usuario , p_clave                                                           | {token, nombre, rol}                                                                        |
| app_logout            | p_token                                                                       | —                                                                                           |
| app_productos         | p_token , p_buscar                                                            | [[{codigo, nombre, categoria, precio, existencia}]]                                         |
| app_clientes          | p_token , p_buscar                                                            | [[{nit, nombre, direccion, telefono}]]                                                      |
| app_registrar_cliente | p_token , p_nit , p_nombre ,<br>p_direccion , p_telefono                      | {id, nit, nombre}                                                                           |
| app_ventas            | p_token , p_desde , p_hasta (aaaa-mm-dd)                                      | [[{numero_factura, fecha, cliente, total}]]                                                 |
| app_detalle_venta     | p_token , p_numero_factura                                                    | [[{producto, cantidad, precio, subtotal}]]                                                  |
| app_resumen           | p_token , p_hoy (fecha local del teléfono), p_existencia_baja (5)             | {ventas_hoy, facturas_hoy, ventas_mes, ventas_7_dias[], top_productos[], existencia_baja[]} |
| app_registrar_factura | p_token , p_nit , p_cliente ,<br>p_fecha , p_detalle ( [{codigo, cantidad}] ) | {numero_factura, total}                                                                     |

Los errores llegan con código HTTP 4xx y `{"message": "..."}` . El mensaje ya está en español y la app lo muestra tal cual.

## 6. App móvil ( `movil/` )

| Paquete | Clases                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| datos   | <code>ConfigSupabase</code> (URL y clave publishable), <code>ApiSupabase</code> (singleton y fachada: llamadas HTTP en un hilo aparte con <code>ExecutorService</code> , respuesta en el hilo principal con <code>Handler</code> ), <code>Sesion</code> (token en <code>SharedPreferences</code> ; nunca guarda la contraseña), <code>CacheCatalogo</code> (copia del catálogo para usar sin conexión)                                                                                                                                                                                                                                                                                                          |
| modelo  | <code>Producto</code> , <code>Cliente</code> , <code>Venta</code> , <code>LineaVenta</code> (detalle y carrito), <code>Resumen</code> (con <code>VentaDia</code> y <code>ProductoVendido</code> )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| vista   | <code>LoginActivity</code> ; <code>MenuActivity</code> (inicio con indicadores y menú en mosaico); <code>ResumenActivity</code> ; <code>NuevaVentaActivity</code> ; <code>DetalleVentaActivity</code> ; <code>ProductosActivity</code> ; <code>ClientesActivity</code> ; <code>NuevoClienteActivity</code> ; <code>VentasActivity</code> ; <code>ActividadBase</code> (abstracta, exige sesión); <code>AdaptadorFilas&lt;T&gt;</code> (tarjetas, genérica); <code>DialogoBusqueda&lt;T&gt;</code> (elegir cliente o producto, genérica); <code>GraficoBarras</code> y <code>GraficoDona</code> (extienden <code>View</code> y dibujan con <code>Canvas</code> en <code>onDraw()</code> ); <code>Formatos</code> |

HTTP ( `URLConnection` ), JSON ( `org.json` ) y gráficos ( `Canvas` ) usan solo lo que trae Android. La **única dependencia** es `androidx.swiperefreshlayout` , el componente oficial para *deslizar para actualizar*.

## Detalles de la app móvil:

| Tema                 | Cómo funciona                                                                                                                                                                                                                                                                                                                                          |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Gráficos</b>      | <code>GraficoBarras</code> escala cada barra al máximo de la semana y escribe su valor encima; destaca el día de hoy. <code>GraficoDona</code> dibuja un arco por producto, proporcional a sus unidades ( <code>drawArc</code> ), y pone el total en el centro. Los colores salen de <code>colores.xml</code> , así que también cambian en modo oscuro |
| <b>Factura móvil</b> | El teléfono envía solo el código y la cantidad de cada producto. <code>app_registrar_factura</code> bloquea la tabla para asignar el número correlativo, toma el precio del catálogo, bloquea cada producto ( <code>SELECT ... FOR UPDATE</code> ), valida la existencia y la descuento. Todo es una transacción: si algo falla, no se guarda nada     |
| <b>Sin conexión</b>  | Cada consulta completa del catálogo se guarda en <code>SharedPreferences</code> ( <code>CacheCatalogo</code> ). Si no hay red, Productos muestra esa copia filtrada, con un aviso y su fecha. Se borra al cerrar sesión                                                                                                                                |
| <b>Modo oscuro</b>   | <code>values-night/colores.xml</code> y <code>values-night/estilos.xml</code> redefinen la paleta y el tema, y Android los aplica solo cuando el sistema está en modo oscuro                                                                                                                                                                           |
| <b>Fechas</b>        | Se usa <code>Calendar</code> y <code>SimpleDateFormat</code> porque <code>java.time</code> requiere Android 8 y la app funciona desde Android 7. El teléfono envía su fecha local ( <code>p_hoy</code> ) porque el servidor trabaja en UTC                                                                                                             |

**Compilar:** abra la carpeta `movil/` en Android Studio y use *Build* → *Build APK(s)*, o desde la terminal ejecute `gradlew assembleDebug` con `JAVA_HOME` apuntando al JDK que trae Android Studio (carpeta `jbr`). El APK queda en `movil/app/build/outputs/apk/debug/` , y la copia lista para instalar está en `entregables/VentasMovil.apk` .

**targetSdk 34:** se deja así a propósito. Desde el API 35, Android obliga el modo de pantalla completa (*edge-to-edge*), que tapanía la barra de título en este diseño con vistas clásicas.

## 7. Instalación y puesta en marcha

### 7.1 Escritorio (desarrollo)

1. Clone el repositorio y abra `sistema-ventas` en NetBeans (proyecto Maven).
2. **Modo local:** ejecute (F6). No se necesita nada más.
3. **Modo nube:** copie `config/bd.properties.ejemplo` como `config/bd.properties` y complete los datos del *Session pooler* de Supabase. Al ejecutar se crean las tablas y funciones.
4. En el primer uso, el sistema pide crear el administrador.

### 7.2 Crear el proyecto en Supabase (una vez)

1. Cree una cuenta en [supabase.com](https://supabase.com), una organización del plan **Free** y un proyecto (región `us-east-1` ).
2. **Connect** → **Session pooler:** anote el host y el usuario para `bd.properties` .
3. **Project Settings** → **API Keys:** copie la *Project URL* y la *publishable key* a `movil/.../ConfigSupabase.java` .
4. Ejecute la app de escritorio una vez: crea todo el esquema.

El plan Free **pausa el proyecto tras 7 días sin actividad**. Se reactiva desde el panel de Supabase con **Restore project**.

## 8. Respaldo y restauración

- **Respaldo:** *Administración* → *Respaldar base de datos* genera un archivo `.db` (SQLite) con todas las tablas. Funciona igual con la base local y con la nube; desde la nube se copian las tablas con `CopiadorBD`.
- **Restaurar en local:** cierre el sistema, reemplace `datos/sistema_ventas.db` por el respaldo y quite `config/bd.properties`.
- **Restaurar en la nube:** con la base de la nube vacía, coloque el respaldo como `datos/sistema_ventas.db` y abra el sistema en modo nube. La importación automática sube los datos.

## 9. Pruebas realizadas

Se ejecutaron pruebas automatizadas contra bases temporales (SQLite) y contra el proyecto real de Supabase. Al terminar, se borraron los datos de prueba.

| Área                    | Qué se verificó                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Esquema                 | 10 tablas (+1 en la nube), llaves foráneas activas, RLS en las 11 tablas                                                                                                                                                                                                                                                                                                                            |
| Migración               | Una base de una versión anterior se actualiza sin perder datos                                                                                                                                                                                                                                                                                                                                      |
| Usuarios                | Validaciones, contraseñas hashadas, intento de inyección SQL rechazado, conversión de PBKDF2 a bcrypt, reglas de administrador                                                                                                                                                                                                                                                                      |
| Inventario              | La compra suma, la factura descuenta, una venta sin existencia se rechaza, eliminar o anular devuelve la existencia                                                                                                                                                                                                                                                                                 |
| Integridad              | No se borran productos ni proveedores con compras; las categorías pasan a SET NULL; los detalles se borran en cascada                                                                                                                                                                                                                                                                               |
| Búsquedas y reportes    | Filtro literal, selección correcta con la tabla filtrada, escape HTML                                                                                                                                                                                                                                                                                                                               |
| Respaldo                | Local ( <code>VACUUM INTO</code> ) y desde la nube a <code>.db</code>                                                                                                                                                                                                                                                                                                                               |
| API móvil               | Login correcto e incorrecto, token falso o cerrado rechazado, consultas, alta de cliente visible en el escritorio, <b>lectura directa de tablas bloqueada (401)</b>                                                                                                                                                                                                                                 |
| Resumen y factura móvil | <code>app_resumen</code> (7 días con ceros, top, existencia baja) y <code>app_registrar_factura</code> (número, precio de la base, descuento de existencia, rechazo por existencia, NIT, repetidos y productos inexistentes). Se probaron <b>contra la nube dentro de una transacción revertida</b> , sin tocar los datos reales. Resumen de escritorio: cálculos verificados con una base temporal |